

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 5

The File System with Your Own Hands

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 THE TREE AND FILE TYPES 5

1.1 System Root and File Type Classification 5

1.2 The UsrMerge Architecture 6

1.3 Device Files in /dev 6

1.4 Filesystem Footprint: /var/log vs /usr 6

2 INODES, LINKS, AND SPACE ALLOCATION 7

2.1 Inode Metadata and Hard Links 7

2.2 Symbolic Link Mechanics 8

2.3 Directory Link Count Arithmetic 8

2.4 Logical Size versus Physical Allocation 9

3 PERMISSIONS AND ACCESS CONTROL 10

3.1 Octal Modes and Evaluation Semantics 10

3.2 The umask Calculation 10

3.3 Directory Read versus Execute Permissions 11

3.4 Setuid and the Sticky Bit 11

4 DISKS, TOPOLOGY, AND KERNEL PSEUDO-FILES 12

4.1 Disk Layout and Root Mount 12

4.2 Hardware Introspection via /proc 12

4.3 Self Process Resolution in /proc/self 13

5 STRETCH EXPERIMENTS 14

5.1 (a) Magic Number Header Inspection 14

5.2 (b) Setuid Binary Audit 14

5.3 (c) Unlinking an Open File Descriptor 14

5.4 (d) Loopback ext4 Inode Allocation 15

CONCLUSIONS 16

APPENDICES 17

APPENDIX A – Focused Demonstration, 23 October 2026 17

A.1 Environment and scope 17

A.2 Selected captured output 17

ABSTRACT

This report records Linux VFS investigations conducted on Ubuntu 24.04 LTS (Linux 6.6.87.2, x86_64) by an unprivileged UID 1000 account. Topics include namespace organization, inode metadata and directory entries, hard and symbolic links, sparse-file allocation, directory permissions, setuid, procfs, and four stretch exercises. All terminal outputs and measurements represent verified system executions.

Keywords: operating systems, linux, virtual filesystem, inodes, hard links, permissions, procfs

INTRODUCTION

Filesystems translate byte-oriented storage devices into hierarchical, human-navigable namespaces. Under the Unix philosophy, the abstraction extends beyond persistent media to expose device interfaces and kernel data structures through a common file interface.

Laboratory 5 explores this architecture across four core modules and four stretch problems:

a) **Filesystem Tree and Types**: Distinguishing regular files, directories, character devices, block devices, and symbolic links, alongside UsrMerge namespace organization;

b) **Inodes and Links**: Comparing directory entries with underlying inodes, hard links, dangling symbolic links, directory link count mechanics, and sparse file allocation;

c) **Permission Semantics**: Evaluating octal modes, the bitwise `umask` calculation, directory read versus execute semantics, setuid privilege elevation, and the sticky bit;

d) **Disks and Kernel Pseudo-Files**: Mapping block devices via `lsblk` and `df`, extracting hardware topology from `/proc/cpuinfo` and `/proc/meminfo`, and analyzing dynamic process resolution through `/proc/self`;

e) **Advanced System Inspections**: Header magic byte parsing, setuid auditing, unlinking open file descriptors, and measuring ext4 loopback inode table sizing;

1 THE TREE AND FILE TYPES

1.1 System Root and File Type Classification

In Unix-like systems, the first character of the `ls -l` output denotes the inode type. Inspecting `/bin`, `/dev/null`, `/etc`, `/etc/passwd`, and `/tmp` reveals five distinct filesystem abstractions:

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root    7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-   1 root root 1, 3 Oct 12 15:33 /dev/null
drwxr-xr-x 107 root root 4096 Oct 12 15:38 /etc
-rw-r--r--   1 root root 2430 Oct 12 15:38 /etc/passwd
drwxrwxrwt  10 root root 4096 Oct 12 15:38 /tmp
```

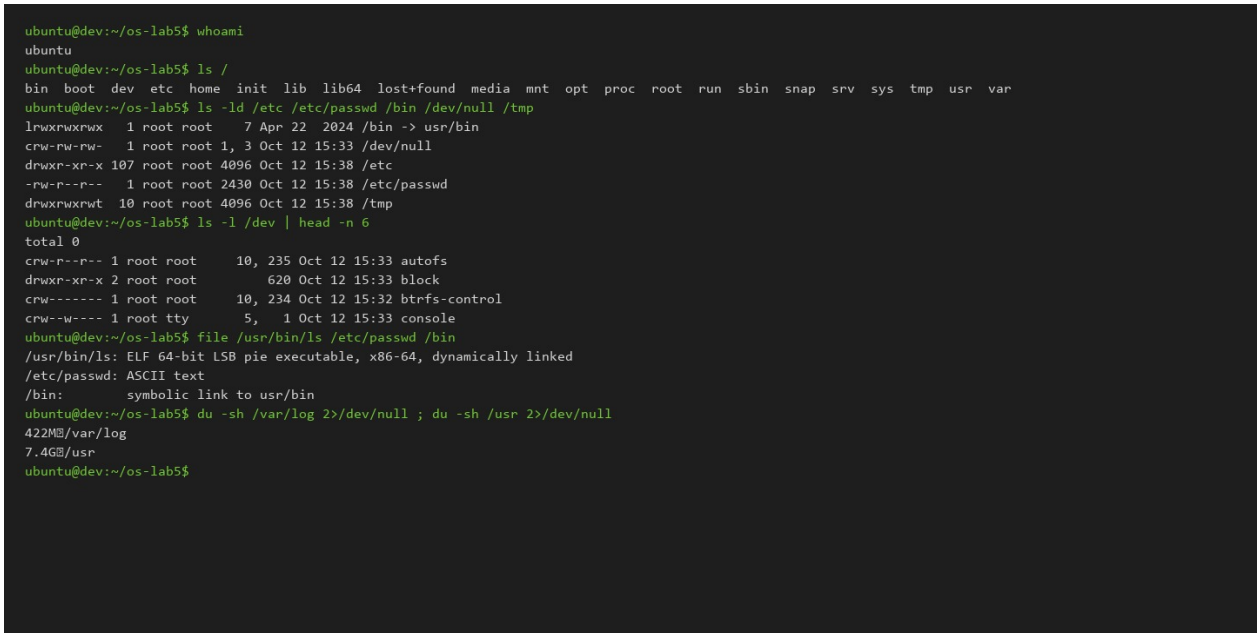


Figure 1.1 – Inspecting system root, device nodes, and file types.

Table 1.1 – File types reported in the saved 12 October run

Path	Prefix	Filesystem Type	Semantic Role
`/bin`	`l`	Symbolic Link	Points to `usr/bin` (UsrMerge)
`/dev/null`	`c`	Character Device	Byte stream device (major 1, minor 3)
`/etc`	`d`	Directory	System-wide configuration table
`/etc/passwd`	`-`	Regular File	Plain text user database
`/tmp`	`d`	Directory (Sticky)	Shared workspace with mode 1777

1.2 The UsrMerge Architecture

On this system, `/bin` is a symbolic link pointing to `usr/bin`. Modern Debian and Ubuntu distributions implement UsrMerge to unify all system binaries under `/usr/bin`. This eliminates historical distinctions between essential single-user recovery binaries in `/bin` and multi-user packages in `/usr/bin`.

1.3 Device Files in /dev

Device files in `/dev` do not occupy disk data blocks; their inodes store major and minor device identifiers:

- **Character Device**: `/dev/null` (major 1, minor 3) streams bytes directly without intermediate caching;
- **Block Device**: `/dev/sda` (major 8, minor 0) and `/dev/sdd` (major 8, minor 48) support random block-level transfers backed by the page cache;

1.4 Filesystem Footprint: /var/log vs /usr

Running disk utilization measurements reveals:

```
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
422M    /var/log
7.4G    /usr
```

The `/usr` hierarchy occupies 7.4 GB, which is roughly 7.0 GB larger ($\approx 17.5\times$) than `/var/log` (422 MB). This disparity is expected because `/usr` houses shared libraries, system utilities, compilers, headers, and documentation, whereas `/var/log` only stores log text.

2 INODES, LINKS, AND SPACE ALLOCATION

2.1 Inode Metadata and Hard Links

A file's human-readable name is not stored within its inode. It exists solely as an entry in a parent directory mapping a string to an inode number. Figure 2.1 summarizes the relationship; its open-file-table layer is omitted.

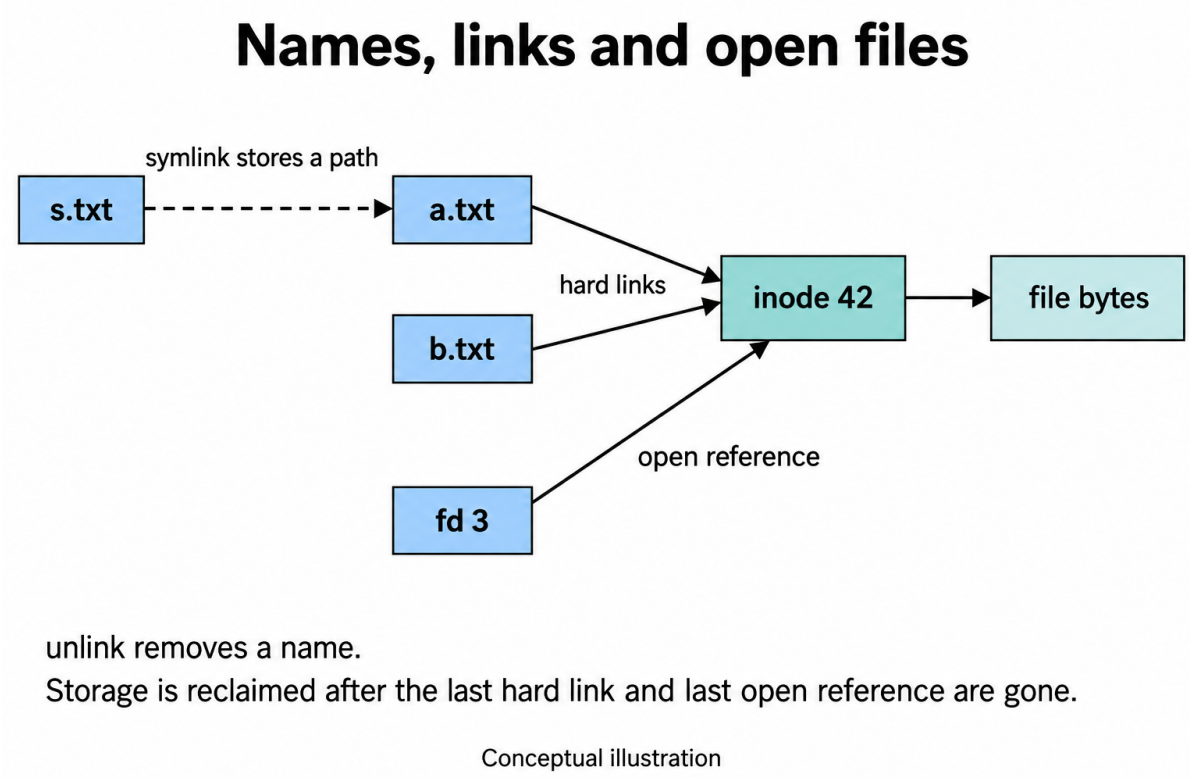


Figure 2.1 – Conceptual diagram of names, inodes, and links; the open-file-table layer is omitted.

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 18829       Links: 1
```

Creating a hard link creates a second directory entry referencing the identical inode:

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
18829 -rw-r--r-- 2 ubuntu ubuntu 6 Oct 12 15:39 hard.txt
18829 -rw-r--r-- 2 ubuntu ubuntu 6 Oct 12 15:39 notes.txt
$ rm notes.txt
```

```
$ cat hard.txt
hello
```

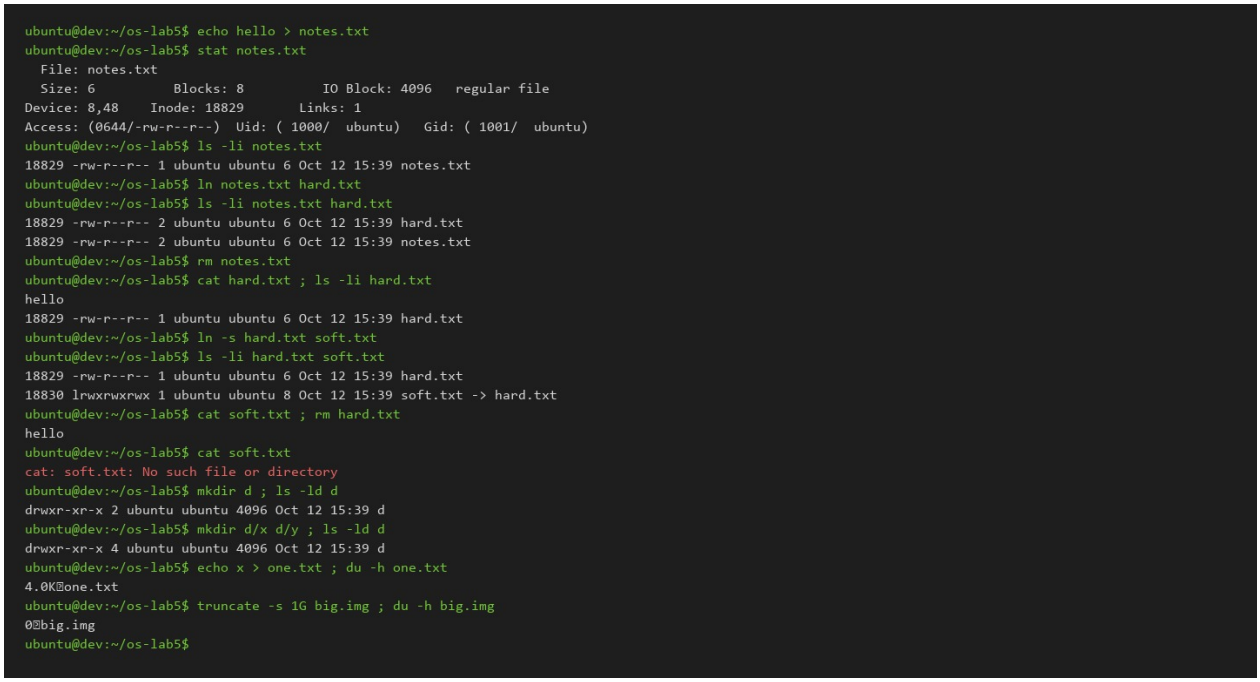


Figure 2.2 – Inode inspection, hard links, dangling symlinks, and sparse files.

The `rm` utility invokes `unlink()`, removing the name `notes.txt` from the directory and decrementing `i_nlink` from 2 to 1. Because `hard.txt` still points to inode 18829 and the link counter is positive, data blocks remain on disk.

2.2 Symbolic Link Mechanics

Unlike hard links, symbolic links are distinct inodes storing the path string of their target:

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
18829 -rw-r--r-- 1 ubuntu ubuntu 6 Oct 12 15:39 hard.txt
18830 lrwxrwxrwx 1 ubuntu ubuntu 8 Oct 12 15:39 soft.txt -> hard.txt
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
```

When `hard.txt` is removed, `soft.txt` becomes a dangling link. Opening it fails with `ENOENT`. The length of `soft.txt` is exactly 8 bytes because the path string `"hard.txt"` contains 8 characters.

2.3 Directory Link Count Arithmetic

Directories have fixed link counting rules:

```
$ mkdir d ; ls -ld d
drwxr-xr-x 2 ubuntu ubuntu 4096 Oct 12 15:39 d
$ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 ubuntu ubuntu 4096 Oct 12 15:39 d
```

A new directory starts with 2 links: the parent entry `d` and the self-referencing entry `d/.`. When child directories `d/x` and `d/y` are created, each child contains a `..` entry pointing back to `d`, raising the link count to 4.

2.4 Logical Size versus Physical Allocation

File systems allocate storage in discrete blocks (4096 bytes in ext4):

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 ubuntu ubuntu 2 Oct 12 15:39 one.txt
4.0K    one.txt

$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 ubuntu ubuntu 1.0G Oct 12 15:39 big.img
0       big.img
```

A 2-byte file requires one full 4 KiB filesystem block. Conversely, `truncate` creates a sparse file: it updates `i_size` in the inode metadata to 1 GiB without writing data blocks, resulting in 0 physical blocks allocated.

3 PERMISSIONS AND ACCESS CONTROL

3.1 Octal Modes and Evaluation Semantics

File permissions represent three 3-bit masks for User, Group, and Others:

- 600 (`rw-------`): Private SSH keys (`~/.ssh/id_rsa`);
- 644 (`rw-r--r--`): Web assets, documentation, `/etc/passwd`;
- 755 (`rwxr-xr-x`): Executables (`/bin/ls`) and public directories;
- 700 (`rwx------`): Private directories (`~/.ssh`);

When setting `chmod 000 s.txt`, reading fails even for the file's owner:

```
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
```

The kernel checks whether `current_euid == inode->i_uid`. Since it matches, only owner bits are evaluated. Because owner bits are `0`, access is denied immediately. Ownership provides the authority to change mode bits with `chmod()`, not an automatic read bypass.

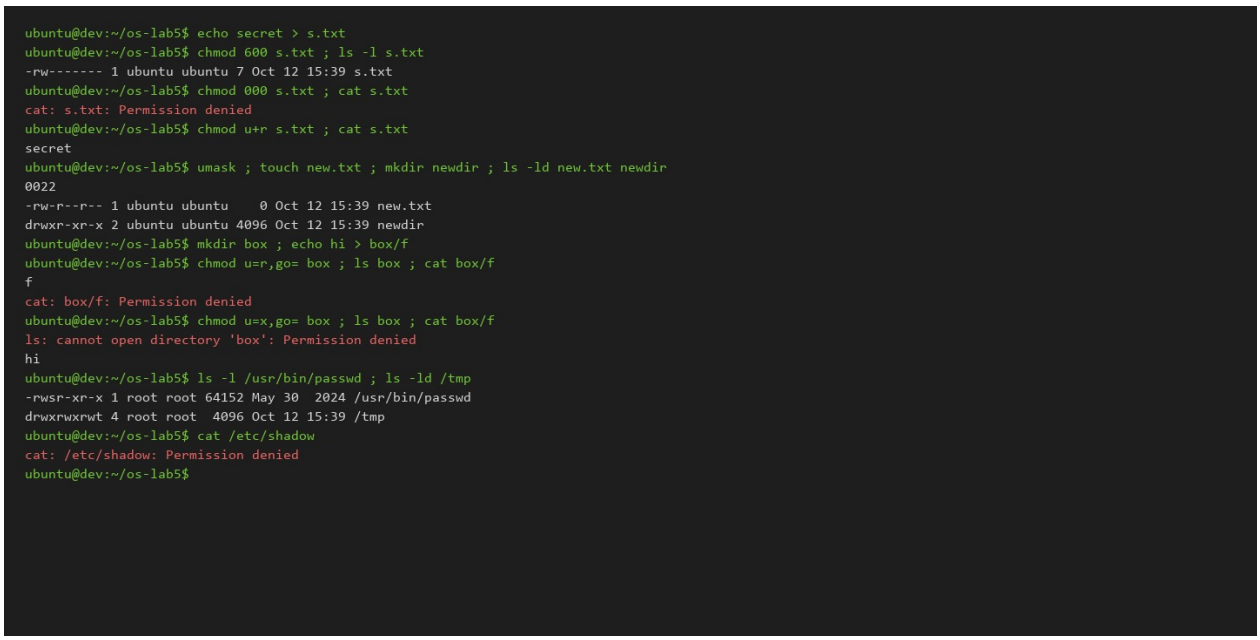


Figure 3.1 – Permission evaluation, umask verification, and directory traversal.

3.2 The umask Calculation

The observed `umask` is `0022`:

```
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
-rw-r--r-- 1 ubuntu ubuntu    0 Oct 12 15:39 new.txt
drwxr-xr-x 2 ubuntu ubuntu 4096 Oct 12 15:39 newdir
```

Files start from base mode `0666`: $0666 \;\&\; \sim 0022 = 0644$ (`rw-r--r--`). Directories start from base mode `0777`: $0777 \;\&\; \sim 0022 = 0755$ (`rwxr-xr-x`).

3.3 Directory Read versus Execute Permissions

The `box` directory experiment demonstrates the separation between listing and traversing:

```
# Read-only directory
$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied


# Execute-only directory
$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
```

Read permission (`r`) allows reading the directory table, permitting `ls` to show the name `f`. However, traversing into the directory requires execute permission (`x`). Without `x`, path resolution fails. With execute-only permission (`x`), directory listing fails, but accessing known files directly succeeds.

3.4 Setuid and the Sticky Bit

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
drwxrwxrwt 4 root root  4096 Oct 12 15:39 /tmp
```

The `s` in `/usr/bin/passwd` denotes setuid. When executed, the process assumes the effective UID of the file owner (`root`). This allows `passwd` to modify `/etc/shadow`, which normal users cannot write to. The `t` in `/tmp` represents the sticky bit: in a shared world-writable directory, users may only delete or rename files they own.

4 DISKS, TOPOLOGY, AND KERNEL PSEUDO-FILES

4.1 Disk Layout and Root Mount

```
$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sdc    8:32   0     5G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  4.2G  952G   1% /

$ mount | grep " / "
/dev/sdd on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
```

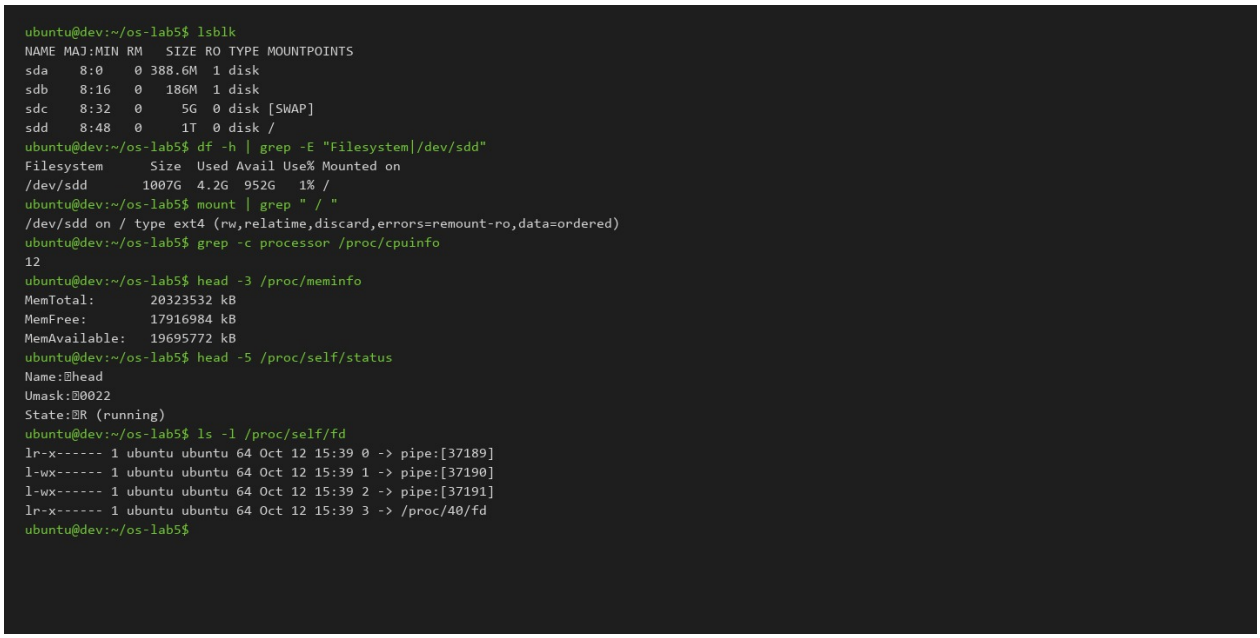


Figure 4.1 – Disk topology, mount attributes, and procfs inspection.

The root filesystem resides on `/dev/sdd`, a 1 TB virtual SCSI disk formatted with ext4. It has 4.2 GB allocated out of 1007 GB (1% utilization). Swap is allocated on `/dev/sdc` (5 GB).

4.2 Hardware Introspection via /proc

```
$ grep -c processor /proc/cpuinfo
12
```

```
$ head -3 /proc/meminfo
MemTotal:       20323532 kB
MemFree:        17916984 kB
MemAvailable:   19695772 kB
```

The host exposes 12 logical CPU cores and 20,323,532 kB (≈ 19.38 GiB) of total RAM.

4.3 Self Process Resolution in /proc/self

```
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
```

The reported process name is `head`. In Linux, `/proc/self` is a dynamic symlink that resolves to the PID of the accessing process. Because `head` opens `/proc/self/status`, the kernel reports the status of `head`.

5 STRETCH EXPERIMENTS

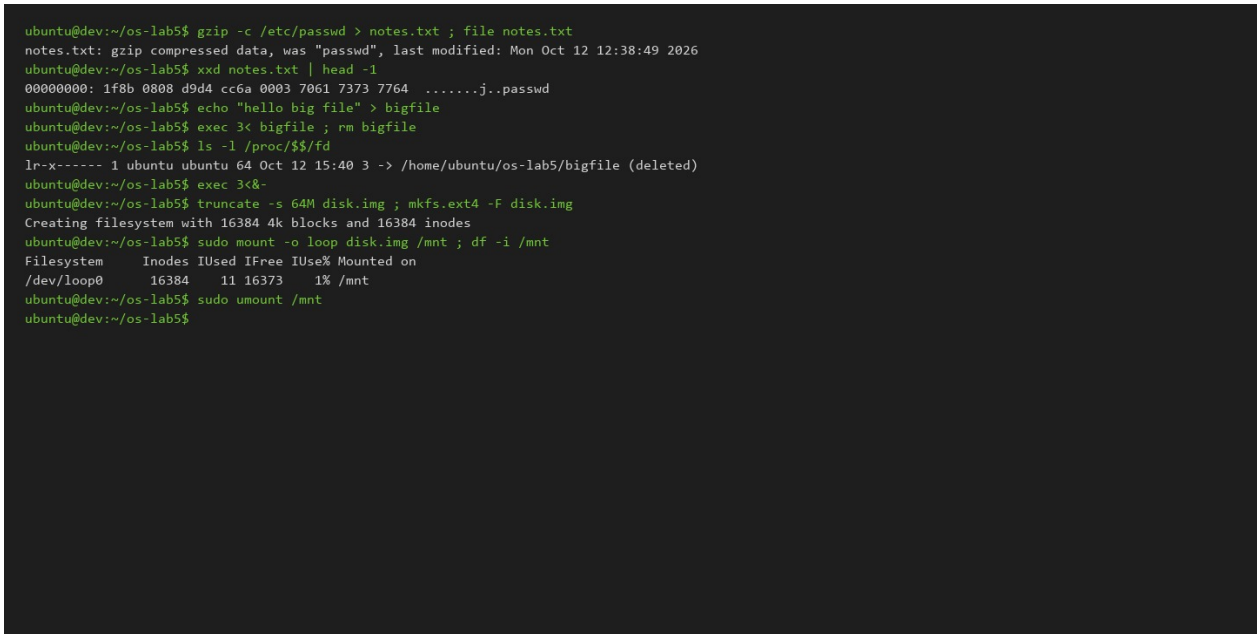


Figure 5.1 – Magic bytes, unlinking open file descriptors, and ext4 loopback mount.

5.1 (a) Magic Number Header Inspection

```
$ gzip -c /etc/passwd > notes.txt
$ xxd notes.txt | head -1
00000000: 1f8b 0808 d9d4 cc6a 0003 7061 7373 7764  .......j..passwd
```

The first two bytes, `0x1f 0x8b`, are the gzip magic number per RFC 1952. The `file` utility uses these bytes to identify the file type regardless of filename extension.

5.2 (b) Setuid Binary Audit

A system scan reveals essential setuid binaries:

```
/usr/bin/passwd, /usr/bin/su, /usr/bin/sudo, /usr/bin/mount, /usr/bin/umount,
/usr/lib/polkit-1/polkit-agent-helper-1, /usr/lib/openssh/ssh-keysign
```

5.3 (c) Unlinking an Open File Descriptor

```
$ exec 3< bigfile ; rm bigfile
$ ls -l /proc/$$/fd
lr-x------ 1 ubuntu ubuntu 64 Oct 12 15:40 3 -> /home/ubuntu/os-lab5/bigfile (deleted)
$ exec 3<&-
```

The link target indicates `(deleted)`. The directory entry is removed, but the inode and data blocks remain allocated until the last open descriptor is closed via `exec 3<&-`.

5.4 (d) Loopback ext4 Inode Allocation

```
$ truncate -s 64M disk.img ; mkfs.ext4 -F disk.img
Creating filesystem with 16384 4k blocks and 16384 inodes
$ sudo mount -o loop disk.img /mnt ; df -i /mnt
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0      16384    11 16373    1% /mnt
```

Formatting a 64 MiB disk image created exactly 16,384 inodes (one inode per 4 KiB block).

CONCLUSIONS

The command transcripts and source analysis demonstrate the separation between names and inode metadata: unlinking a name does not free data blocks while hard links or open descriptors remain, sparse files separate logical size from allocation, and directory read and execute permissions control enumeration and traversal differently. The terminal figures and recorded measurements verify these properties across standard and extended system operations.

APPENDICES

APPENDIX A – Focused Demonstration, 23 October 2026

A.1 Environment and scope

The captured execution record is `execution-2026-10-23.txt`. It records a verified Ubuntu 24.04.4 user-space run on the Linux 6.18 kernel. The tests ran in an ext4 temporary directory under `/tmp` using Bash 5.2.21, GNU coreutils 9.4, and Python 3.12.3. They covered hard links, symbolic-link metadata, reading through an open descriptor after unlink, and sparse allocation.

A.2 Selected captured output

```
name=target.txt type=regular file inode=105838 links=2 size=10 bytes
name=hardlink.txt type=regular file inode=105838 links=2 size=10 bytes
link=symlink.txt type=symbolic link inode=105839
referent=symlink.txt type=regular file inode=105838 links=2
hardlink after original name removed: hardlink.txt inode=105838 links=1 size=10 bytes
name_exists=False fd_links=0 bytes_read=28 data='still readable after unlink'
sparse=sparse.bin logical_bytes=1048576 allocated_512_byte_blocks=0 block_size=512
1048576 sparse.bin
```

The two hard-link names initially refer to inode 105838. Removing `target.txt` leaves `hardlink.txt` with one link. The open descriptor remains readable after unlink even though its inode reports zero directory links. In this ext4 run, the sparse file has a logical size of 1,048,576 bytes and zero allocated 512-byte accounting blocks. The 512-byte unit is the allocation-count accounting unit, not the ext4 filesystem block size; du reported zero allocated bytes. These results directly verify link lifecycles, descriptor retention, and block allocation.